Project Title: Firewall + IDS Mini Setup

Overview

This project demonstrates a layered network security setup by combining a firewall with Intrusion Detection Systems (IDS). The firewall was configured using iptables to allow or block specific network traffic, while Snort and Suricata were deployed to monitor network activity and detect suspicious behavior. Nmap was used to verify the effectiveness of the firewall rules.

Objective

The purpose of the experiment is the installation and configuration of the Apache2 web server in the Kali Linux operating system. In addition, an explanation will be provided of how firewalls and intrusion detection systems operate in order to provide security in the network environment. The main task of the experiment will be to configure the firewall by means of the iptables in such a way that it will only allow the use of secure protocols of HTTP, HTTPS, and SSH. On the other hand, FTP and Telnet protocols will be prohibited. Furthermore, the experiment will include the installation and configuration of the Snort and Suricata systems, which serve as network-based intrusion detection systems.

Also, the goal of the experiment will be the testing of the efficiency of the configured firewall by means of Nmap and the ability of Snort and Suricata systems to detect malicious activities in the network.

Introduction

Network security is another element of the current information system. The infrastructure of organizations uses several security levels instead of one method to secure the environment. There are several security technologies; however, two of them are the firewall and the Intrusion Detection System (IDS).

The firewall is responsible for controlling the flow of network traffic based on some security policies. This technology prevents any unauthorized access and allows the traffic only from the trusted sources.

The Intrusion Detection System controls the network traffic and analyzes packets for any malicious activity, attacks, and violations of security policies. It should be mentioned that

unlike the firewall, the IDS is not responsible for blocking the traffic; it only gives an alert about the violation of the policy.

In this experiment, iptables were used for configuring the firewall for allowing HTTP, HTTPS, and SSH traffic but not for the other protocols like FTP and Telnet. In addition, Snort and Suricata tools were used for monitoring the traffic and giving alerts about the violation of the policies. Finally, Nmap was used for checking the firewall rules and services.

Tool Used

Tool	**Purpose**
kali	Operating System
Apache2	Web Server for testing firewall rules
iptables	Linux Firewall Configuration
Snort	Network Instruction Detection System (IDS)
Suricata	Advanced Network IDS/IPS Engine

Technical Background

Apache2 Web server

One of the popular open-source web servers is Apache2. It runs the web pages and serves HTTP service to the client. In this particular experiment, Apache2 was set up to serve active HTTP service for firewall testing.

Iptables Firewall

iptables is the Linux kernel firewall utility used to create rules that control incoming and outgoing network traffic.

Firewall rules generally perform actions such as:

- ACCEPT
- DROP
- REJECT

In this experiment:

- HTTP (Port 80) → Allowed
- HTTPS (Port 443) → Allowed
- SSH (Port 22) → Allowed

- FTP (Port 21) → Blocked
- Telnet (Port 23) → Blocked

This configuration demonstrates how firewalls restrict unnecessary services while allowing legitimate communication.

Snort IDS

Snort is an open-source Network Intrusion Detection System capable of performing real-time traffic analysis and packet inspection.

Its primary functions include:

- Packet sniffing
- Protocol analysis
- Intrusion detection
- Rule-based alert generation
- Network traffic monitoring

Snort analyses packets using predefined rule sets and generates alerts when suspicious traffic matches known attack signatures.

Suricata IDS

Suricata is a high-performance open-source IDS/IPS engine capable of multi-threaded packet inspection.

Its capabilities include:

- Deep Packet Inspection (DPI)
- Intrusion Detection
- Intrusion Prevention
- Protocol Identification
- File Inspection
- Network Traffic Analysis
- Real-time Alert Generation

Suricata stores alerts and logs in files such as:

- eve.json
- fast.log
- stats.log
- suricata.log

These logs assist security analysts in forensic investigations and incident response.

Nmap

Nmap (Network Mapper) is an open-source network scanning tool used for discovering hosts, open ports, running services, and operating systems.

In this experiment, Nmap verified that firewall rules were functioning correctly by confirming that:

- HTTP remained accessible.
- Blocked services were filtered.
- The Apache server responded successfully.

Methodology/Procedure

Step1: Start Apache Web Server

The Apache2 service was started and enabled to ensure that the web server remained active after reboot.

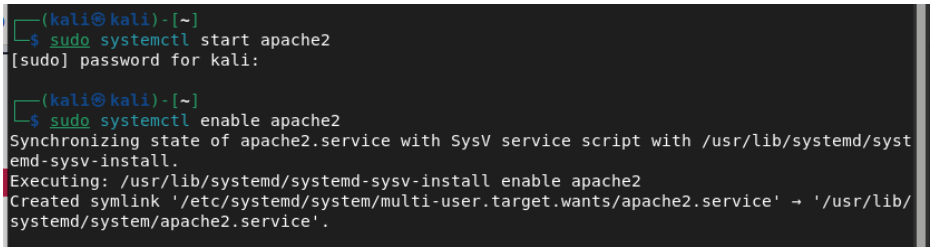


Fig7.1: Apache service started and enabled

Step2: Verify Apache Web Server

The successful loading of the Apache2 Debian Default Page confirmed that the web server was functioning correctly.

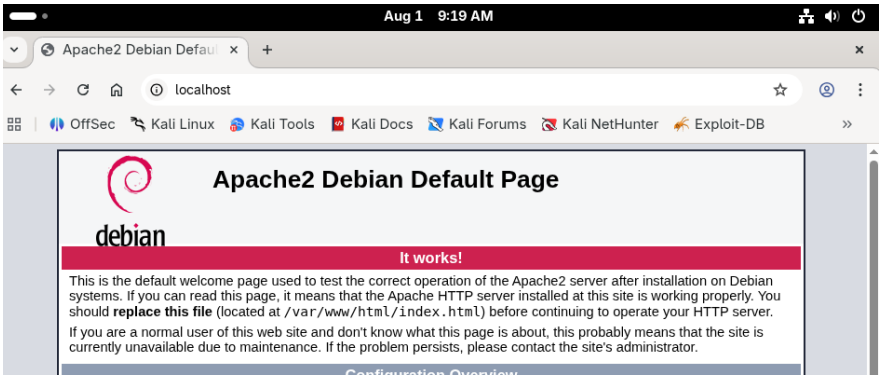


Fig7.2: Apache Default Page.

Step 3: Install Snort IDS

Snort IDS was installed using the package manager.

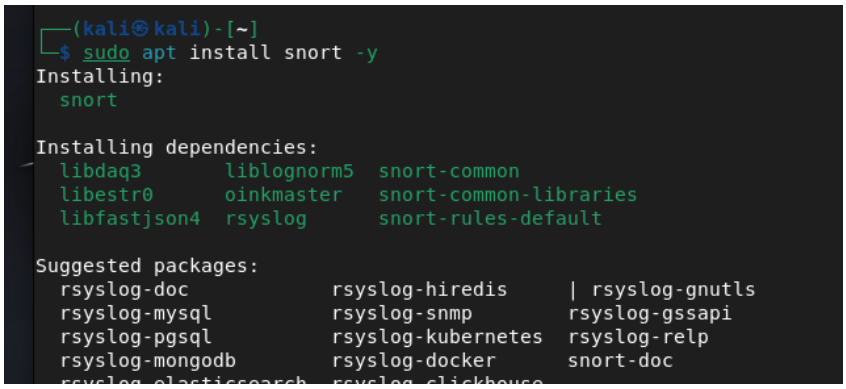


Fig7.3: Snort installation.

Step 4: Identify Network Interface

The active network interface was identified. The interface eth1 with its assigned IP address was selected for IDS monitoring.

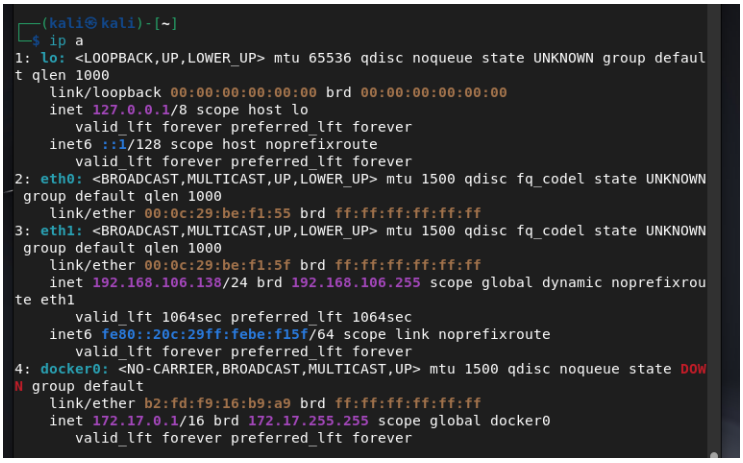


Fig7.4: Network interface information.

Step 5: Start Snort IDS

Snort was launched in packet monitoring mode. Snort successfully loaded its configuration and began monitoring network packets.

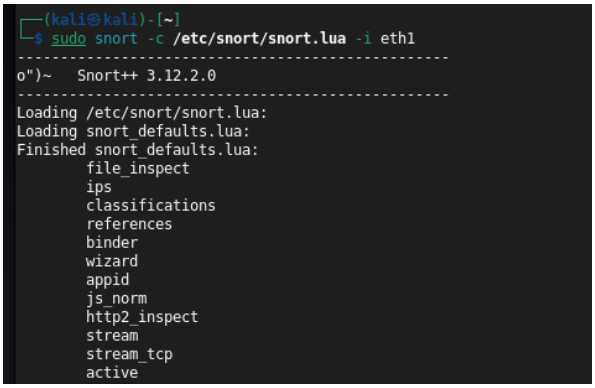
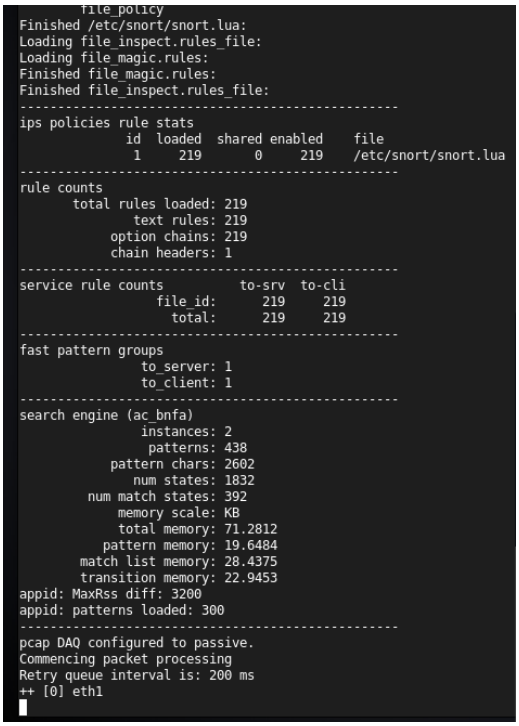


Fig7.5: Snort startup.

Fig7.6: Snort actively monitoring network traffic.



Step 6: Display Existing Firewall Rules

The current firewall configuration was checked before adding new rules.

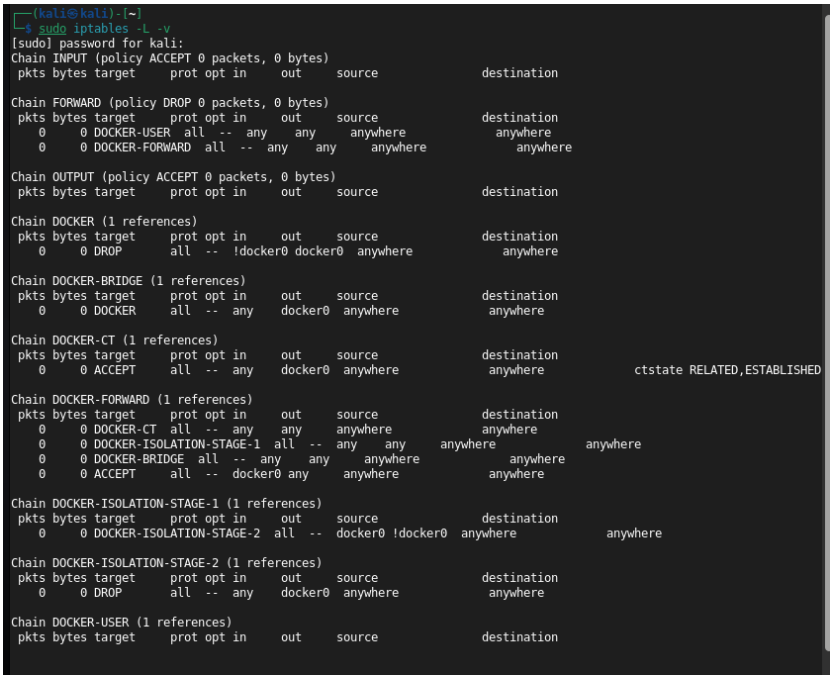
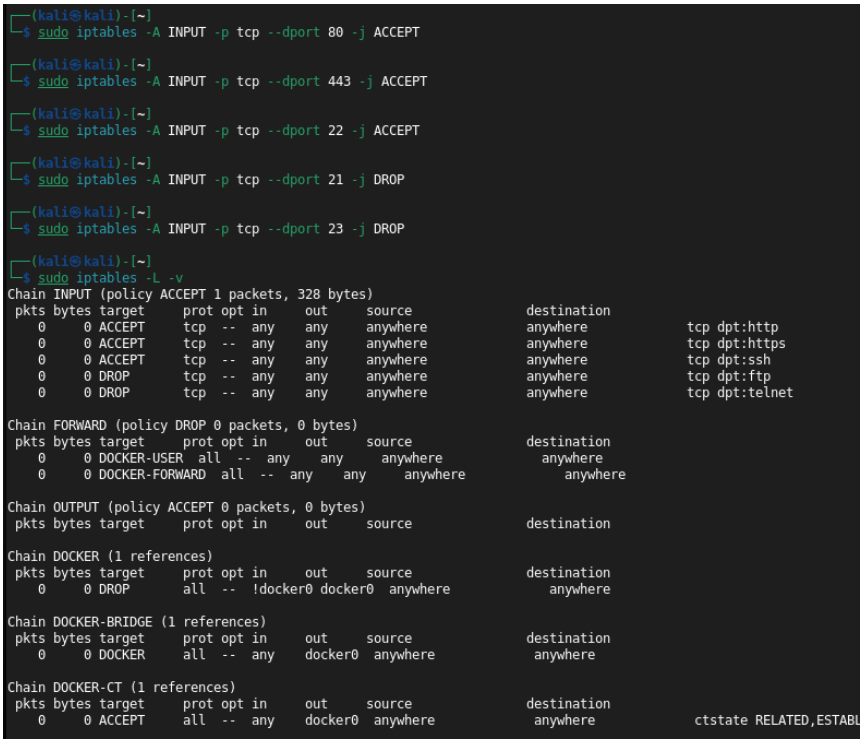


Fig7.7: Initial firewall rules.

Step 7: Configure Firewall Rules

The firewall was configured to allow secure services and block insecure ones.

Fig7.8: Firewall rule configuration.



Step 8: Verify Firewall Rules

The firewall configuration was verified.

The output confirmed:

- HTTP allowed
- HTTPS allowed
- SSH allowed
- FTP blocked
- Telnet blocked

Fig7.9: Firewall verification.

Step 9: Verify Using Nmap

Nmap scans were executed to verify firewall behavior.

The scan confirmed:

- Port 80 open
- Blocked ports filtered

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sS 192.168.106.138
Starting Nmap 7.98 ( https://nmap.org ) at 2026-08-01 13:01 -0400
Nmap scan report for 192.168.106.138
Host is up (0.0076s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT   STATE SERVICE
80/tcp open  http

Nmap done: 1 IP address (1 host up) scanned in 7.21 seconds

┌──(kali㉿kali)-[~]
└─$ sudo nmap -A 192.168.106.138
Starting Nmap 7.98 ( https://nmap.org ) at 2026-08-01 13:02 -0400
Nmap scan report for 192.168.106.138
Host is up (0.00054s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.66 ((Debian))
|_http-title: Apache2 Debian Default Page: It works
|_http-server-header: Apache/2.4.66 (Debian)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running: Linux 2.6.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:2.6.32 cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:6
OS details: Linux 2.6.32, Linux 5.0 - 6.2
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.84 seconds
```

Fig7.10: Nmap verification.

Step 10: Install Suricata IDS

Suricata was installed.

Fig7.11: Suricata installation

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install suricata -y
Upgrading:
  hwloc  libhwloc-plugins  libhwloc15

Installing:
  suricata

Installing dependencies:
  isa-support        librte-eal26      librte-mempool26   librte-sched26
  libfdt1            librte-ethdev26   librte-meter26     librte-telemetry26
  libhyperscan5      librte-hash26     librte-net-bond26  libxdp1
  libnetfilter-log1  librte-ip-frag26  librte-net26       sse3-support
  librte-argparse26  librte-kvargs26   librte-pci26       sse4.2-support
  librte-bus-pci26   librte-log26      librte-rcu26       suricata-update
  librte-bus-vdev26  librte-mbuf26     librte-ring26

Suggested packages:
  libtcmalloc-minimal4

Summary:
  Upgrading: 3, Installing: 28, Removing: 0, Not Upgrading: 1814
  Download size: 8,477 kB
  Space needed: 37.1 MB / 1,247 MB available
```

Step 11: Update Suricata Rules

The latest IDS detection rules were downloaded.

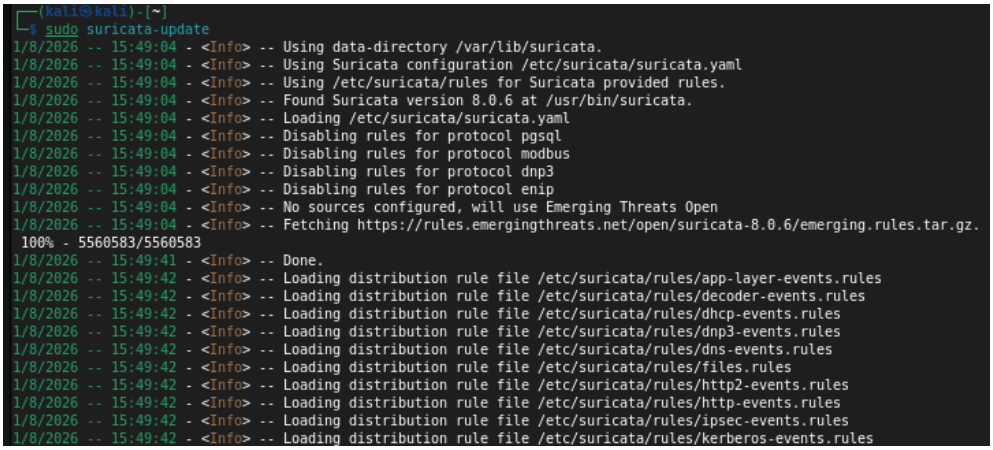


Fig7.12: Suricata rule update.

Step 12: Validate Configuration

The Suricata configuration was tested before execution.

The configuration test completed successfully without errors.

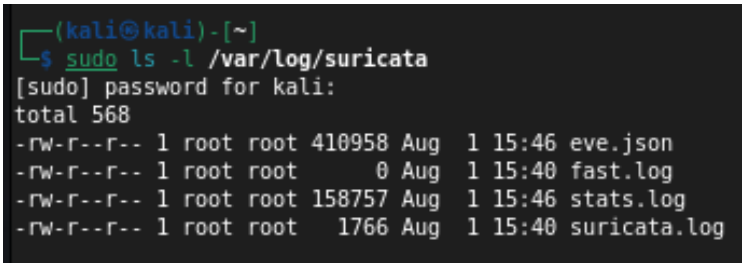


Fig7.13: Configuration validation.

Step 13: Start Suricata

Suricata was launched on the selected interface. Suricata began monitoring network traffic.

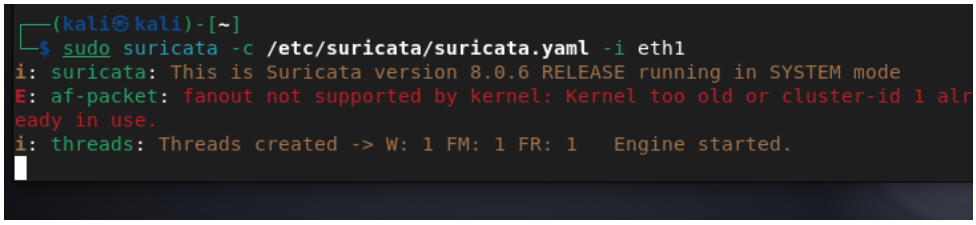


Fig7.14: Suricata monitoring.

Step 14: IDS Alert Generation

Network traffic generated alerts detected by Suricata.

Example alert:
ARP Spoof

This confirmed that the IDS successfully detected suspicious network activity.

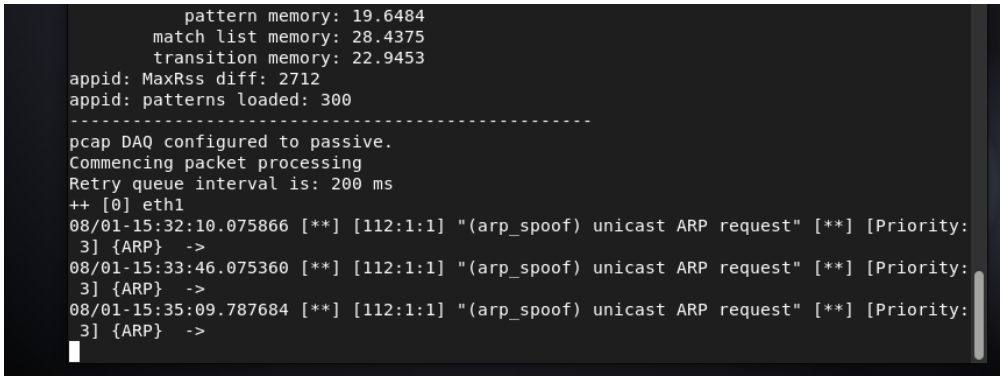


Fig7.15: Suricata detecting ARP spoof alerts.

Step 15: Verify Log Files

Generated log files were verified.

Important logs included:

- eve.json
- fast.log
- stats.log
- suricata.log

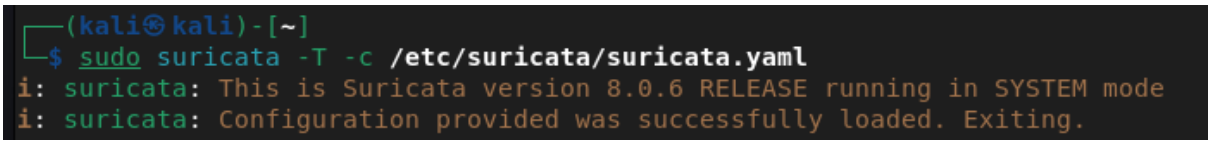


Fig7.16: Suricata log verification.

Analysis

In regard to the firewall, the access control was successfully implemented since only necessary services were allowed, and the vulnerable protocols like FTP and Telnet were prohibited. Nmap scan indicated that only necessary ports were open.

Snort started working properly, analysing the network live traffic in real-time mode. In addition, Suricata made the monitoring process more efficient, analysing the network traffic using up-to-date detection rules and reporting on any suspicious activity such as ARP spoofing.

The obtained logs showed that Suricata was logging all the events in the network which was very useful for forensic analysis and incident investigations. Thus, the two-level security system was successfully implemented.

Observation

- Apache2 service launched successfully and served the default webpage.
- Snort launched and analyzed the traffic without errors.
- The active network interface (eth1) was identified successfully.
- Firewall successfully allowed traffic for HTTP, HTTPS, and SSH.
- Traffic for FTP and Telnet was successfully blocked according to rules.
- Nmap scan confirmed that traffic on blocked ports is filtered, while HTTP port is available.
- Suricata was installed successfully and rule base was updated.
- Configuration validation was performed successfully.
- Suricata found the traffic related to ARP spoofing and generated alerts.
- Log files (eve.json, fast.log, stats.log, and suricata.log) were successfully created.

Result

The experiment was conducted successfully because the multilayered network security system, which was built using iptables, Snort, and Suricata, was successfully implemented in the Kali Linux environment. The firewall was configured successfully because it allowed only the required services to be available and filtered all other services, which could potentially cause threats to the network. Both Snort and Suricata successfully processed the network traffic and found the suspicious behavior and produced security alerts and logs.